NJIT New Jersey Institute of Technology

Digital Logic Analyzer

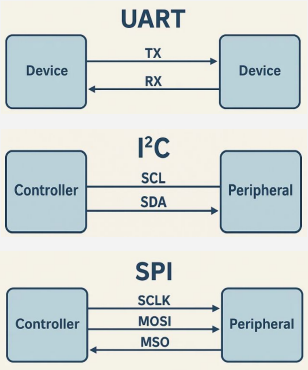
By: Arnav Revankar, Gonzalo Villanueva, Raghav Bharathan, William Dougherty, Bryan Galecio

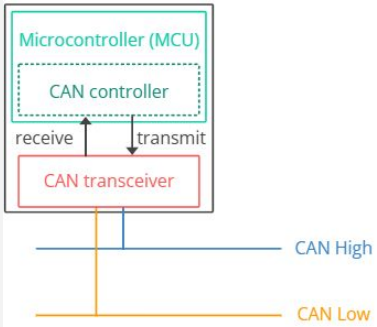
Agenda

- Problem Statement
- Project Objectives & Specifications
- Components & Cost
- System Overview
- Code Overview
- GUI Overview
- Conclusion

Problem Statement

- Digital Communication
  - UART, SPI, I²C, CAN
- Challenges within Embedded Systems
  - Different protocols use varying frame structures and bit lengths, requiring precise decoding
  - Accurate communication depends on strict clocking or synchronization between transmitter and receiver
  - Bit mismatches, noise, or misaligned decoding can cause undetected data corruption
- Digital Logic Analyzer (SOLUTION)
  - Tool that captures and visualizes digital signals over time





Project Objectives & Specifications

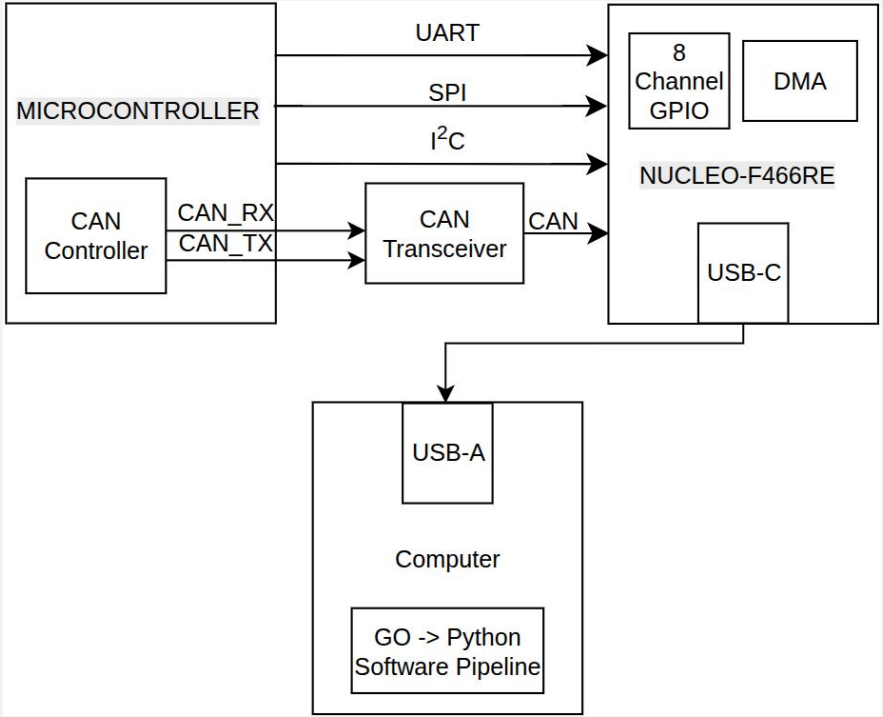
- Build an 8-channel Digital Logic Analyzer using GPIO inputs
- Use internal timer & DMA to sample signals at a fixed rate, offloading the CPU
- Use double buffering to continuously capture and store data without sample loss
- Stream data to a PC over USB with basic error detection (sequence numbers + checksum)
- Develop a PC GUI for real-time waveform visualization and basic protocol decoding (UART, SPI, I2C, CAN)
- Design a PCB layout optimized for low noise, higher sample rate, and system compatibility

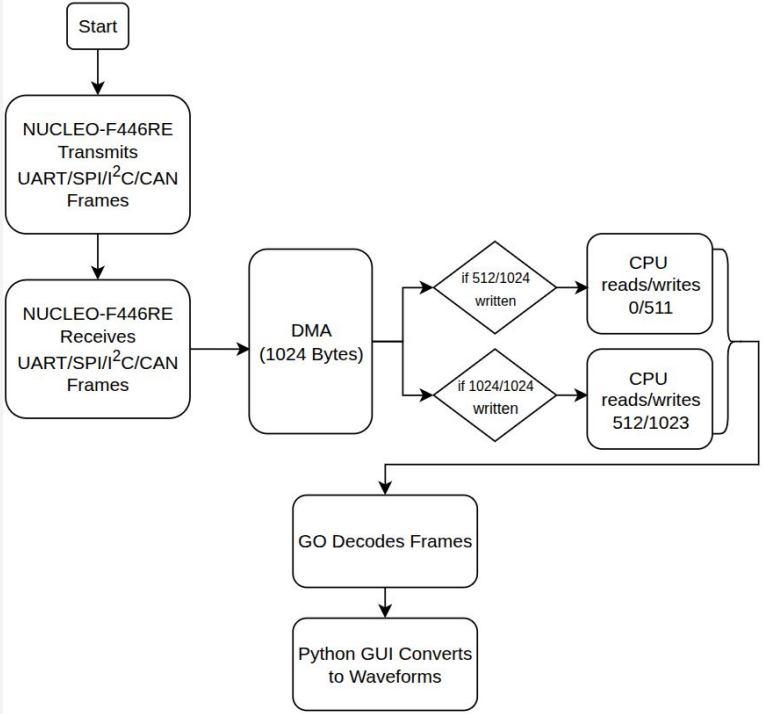
Components & Cost

Component	Quantity	Cost
NUCLEO-F446RE	1	$14.53
SN65HVD230 Can Bus Transceiver	1	$2.52
TXS0108E 8-Bit Bi-Directional Level-Shifter	1	$1.69
USB-C Socket	1	$1.32
Custom PCB	1	$9

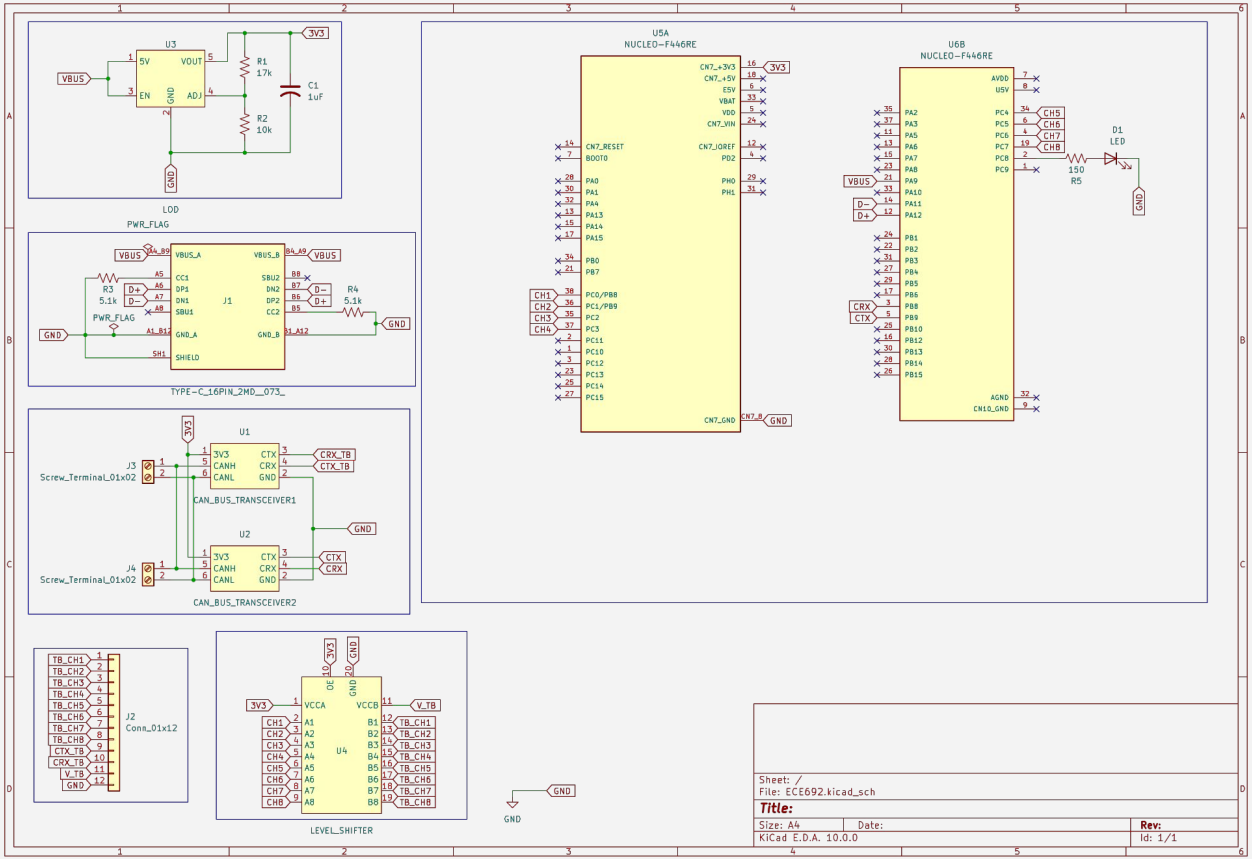
Total Cost: $29.06

System Overview (Block Diagram & Software Pipeline)

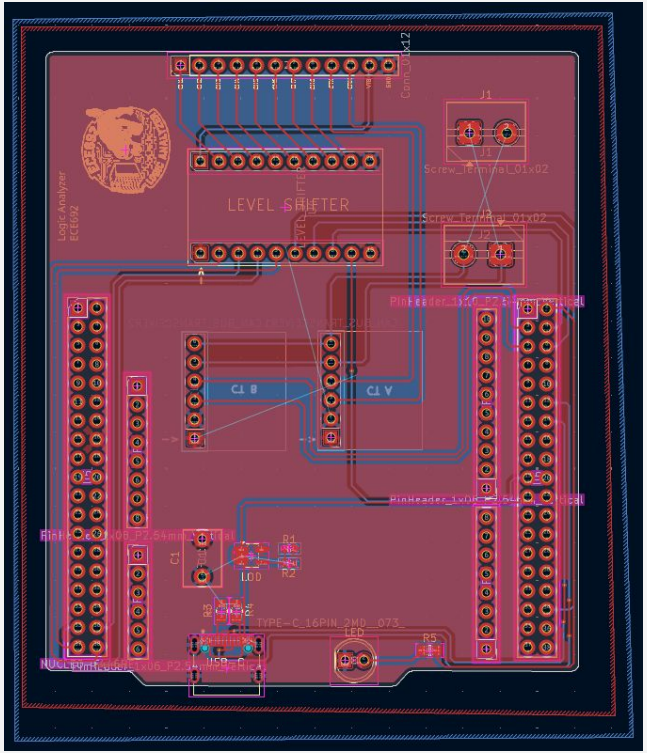


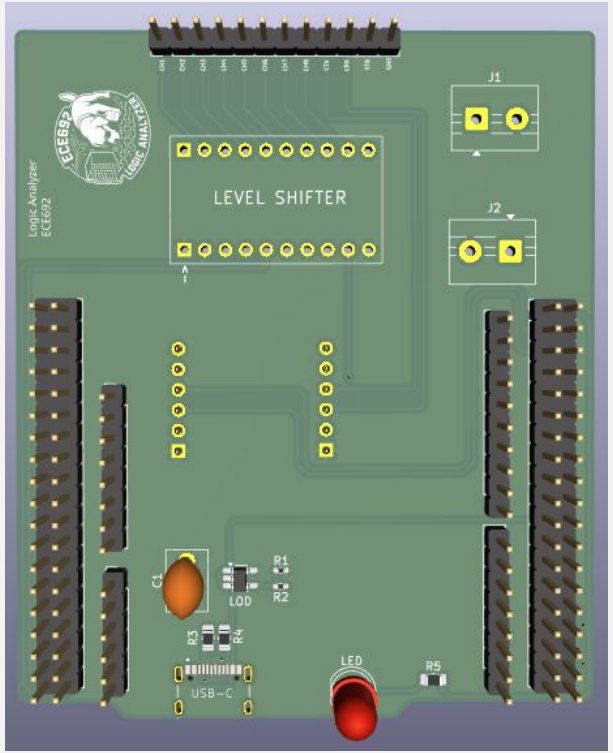


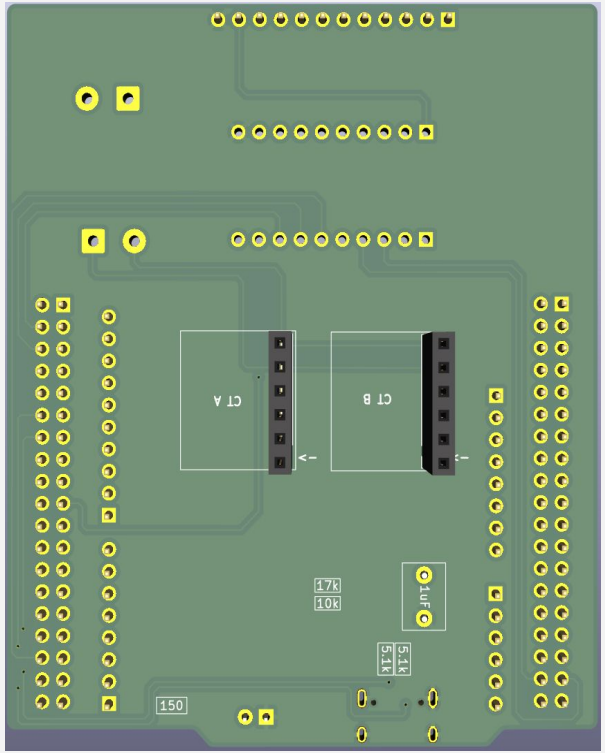
System Overview (Schematic)



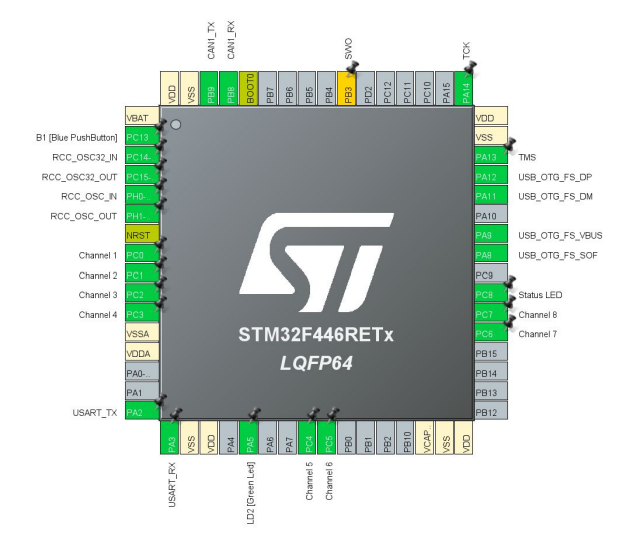
System Overview (PCB Diagram)







STM32 CubeMX Configuration



System Clock

- HSE (8MHz crystal), PLL: PLLM=4, PLLN=168, PLLP=2 → 168MHz HCLK (max)
- APB1 = 42MHz (/4), APB2 = 84 MHz (/2)

TIM1 – Sampling Clock

- Prescaler=99, Period=167 → 10kHz sample rate
- Master output trigger (TRGO) triggers the update for the DMA.

DMA2

- Source: GPIOC→IDR | Destination: dma_buf
- Circular, byte→byte, High priority
- Half and Full transfer interrupts enabled

NVIC Priorities

- DMA2 Stream5: Priority 0 (highest)
- USB: Priority 1
- bxCAN RX0: Priority 1

bxCAN1

- PSC = 6, BS1 = 11TQ, BS2 = 2TQ → 500kbps, PB8/PB9, RX FIFO0 interrupt is enabled

STM32 Sniffer Firmware Pseudocode Overview

```
CONFIGURE:
        GPIO PC0 - PC7: 8-channel digital inputs
        TIM1: 10kHz sampling clock
        DMA2 Stream5: Circular buffer to store samples
        USART2: 115200 baud (ST-link to PC)

START TIM1
START DMA
        Source: GPIOC input register (all 8 channels)
        Destination: Ping-Pong buffer with 2 512 byte Data Chunks = 1024 bytes
        When the DMA is half-full: flag send_first_half = 1
        When the DMA is full: flag send_second_half = 1
LOOP:
   IF first half of DMA buffer is ready:
      SEND first half of samples to PC
   IF second half of DMA buffer is ready:
      SEND second half of samples to PC
```

STM32 Sniffer Firmware bxCAN Pseudocode Overview

CONFIGURE bxCAN1:

Filter: Accept-all (mask = 0000, bank = 0)

Mode: Normal

START CAN peripheral

Enable RX0 FIFO Interrupt

ON INTERRUPT - if frame received:

READ from FIFO:

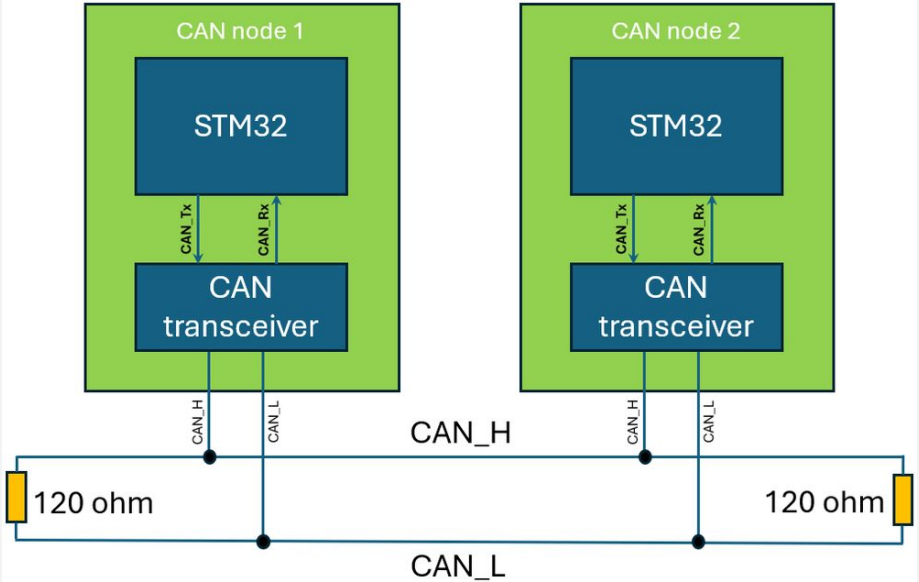
StdId → 11-bit CAN ID

DLC → data length (0–8 bytes)

Data[] → payload

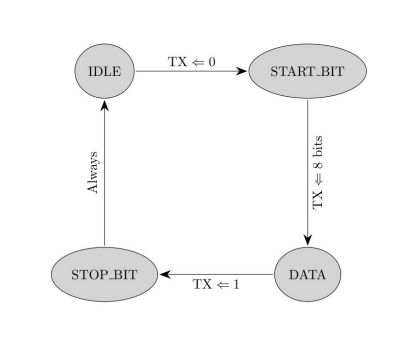
BUILD packet: [0xCC][0xDD][seq][ID_H][ID_L][DLC][data][XOR checksum]

TRANSMIT over USART2 → PC



Decoding the Protocols

- For packet decoding, we used Go.
- Packets arrive from the STM via a COM port (COMx or /dev/ttyx) in custom 516 byte format:
  - [Header 0xAABB] [Sequence num (0 – 255)] [512 Channel Samples] [XOR Checksum]
- The Go decoder reads these packets off the COM port, and will:
  - Validate the checksum (XOR all received bytes)
  - Validate the sequence number. If invalid then a packet was dropped.
  - Reconstruct the channel waveforms, and decode the chosen signal protocol.
- Currently, UART, I2C, and SPI are supported, and tested with simulated packets
- However, only UART was tested, due to sample rate constraints.
- Waveform data and decoded packets then sent to Python GUI via network socket.



GUI Overview

- Uses:
  - Python
    - PyQT5
    - PyQTGraph
    - Numpy
    - Time

Pseudocode:

- Captures COM port data through Go (transport.py)
- Reads packets / parses them (packets.py)
- Plots the waveforms next to their corresponding labels(display.py)
- Opens an application with logical analyzer tool (gui.py)

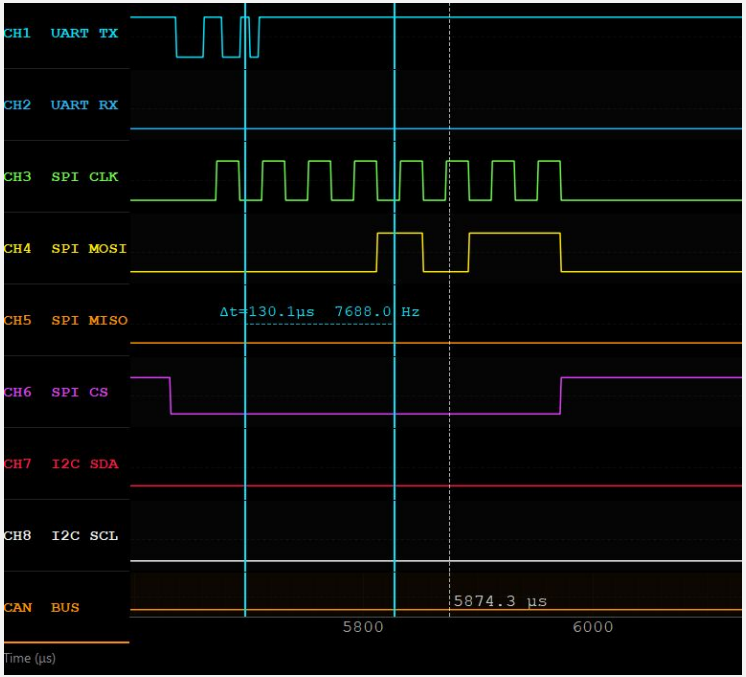
GUI Overview (continued)

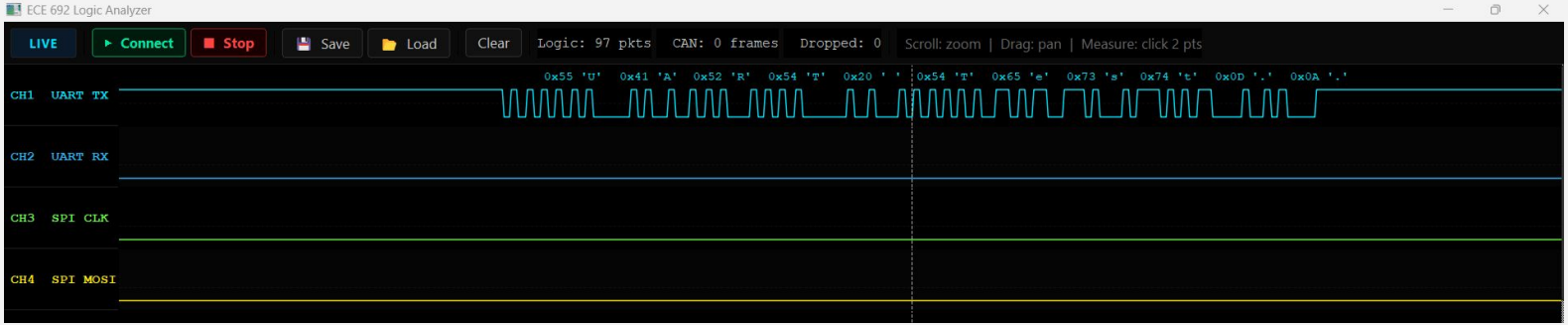
Two main regions:

- **Top:**
  - Toolbar
  - Waveforms / Labels
- **Bottom:**
  - Can Frame Log
  - Settings Panel
    - Protocol Decode Panel
    - Connection Panel
  - Status Bar

GUI Overview (continued)

- **Waveforms:**
  - Labels on the left
  - Corresponding Waveforms on the right
- Able to measure distance between two points
- Zoom/Pan capabilities





Conclusion

- Challenges
  - ST-Link Bottleneck During Prototyping
    - Low throughput caused data transfer issues and incorrect decoding; resolved with custom PCB + improved USB interface
- Future Improvements
  - Switch from ST-Link USART to USB CDC from the PCB connector. This will enable 1MHz sampling and 115200 baud rate for UART decoding, which would reduce noise and provide better accuracy.
  - Improve the decoded frame sizing in the GUI and potentially move one of the CAN transceivers outside the PCB for CANH/CANL compatibility.
  - Implement compatibility for protocols outside the scope of what we have so far (UART, SPI, I^2C, CAN) and also with Sigrok and Pulseview's byte structure format.

Questions?